

Cellivo Development Work Allocation

How the Cellivo components are divided between five developers, with timeline, dependencies and team rules.

Based on Cellivo SRS v1.2 and FRS v1.2

Team 5 developers

Duration About 32 weeks

Date 28 September 2026

1. How the work is split

Each developer owns one area from database to screen (full-stack), so one person is responsible for each feature working end to end. Areas were chosen so the work is roughly equal (about 30 developer-weeks each) and so the parts that others depend on (permissions, UI components, stock, ledger, notifications) are built first. Function IDs (F-xx, A-xx) and requirement IDs refer to the FRS and SRS.

Developer	Area	Main functions	Screens (from UI design)
Developer 1	Tech lead · Platform core and security	Project setup; Multi-tenancy; F-03 Login and security; F-04 Users, roles, permissions; Audit log; F-19 Shop settings; F-16 Branches (core); F-15 Staff and payroll	Login, Roles and permissions
Developer 2	Account portal, billing and platform admin	Shared UI library; F-01 Registration and trial; F-02 Subscription and billing; F-20 Account portal home; Plan limits service; A-01 to A-16 Admin dashboard; Public website hooks	Sign-up, Account portal (new and active), Admin overview, tenants, tenant profile, plans
Developer 3	POS, sales and money	F-09 POS sale; Printing; F-10 Returns, trade-ins, wholesale; F-11 Credit and installments; F-14 Cash drawer and finance; Ledger service; Loyalty (earn/redeem)	POS billing, Payment, Cash drawer close
Developer 4	Inventory, IMEI and purchasing	F-06 Products and inventory; F-07 IMEI tracking; Barcode labels; F-08 Purchasing and suppliers; F-16 Stock transfers; Stock service; INT-01 WooCommerce sync	Inventory, IMEI search and history, GRN

Developer	Area	Main functions	Screens (from UI design)
Developer 5	Repairs, customers, notifications and reports	F-12 Repairs; Public repair status page; F-13 Customers; F-18 Notifications; F-05 Shop dashboard; F-17 Reports	Dashboard, Repair board, New repair, Customer profile, Reports, Mobile views

Replace "Developer 1–5" with names. Developer 1 should be the most experienced person, because the platform core and security affect everyone.

2. Detailed allocation

Developer 1 — Tech lead • Platform core and security

Architecture, multi-tenant foundation, login and security, roles and permissions, audit log, shop settings, branches, staff and payroll, DevOps.

Component	What to build	SRS reference
Project setup	Repository, coding standards, CI/CD, staging and production environments, database migrations, error monitoring	—
Multi-tenancy	Tenant isolation layer (every query scoped by tenant), tenant provisioning service used by sign-up	6, design constraints
F-03 Login and security	Login, password reset, 2FA, IP allow-list, device approval, session timeout, account lock	SEC-01 to 03, 06 to 10
F-04 Users, roles, permissions	User invites, default and custom roles, permission grid, special permissions (view cost, override price, void)	SEC-04, 05, HR-01
Audit log	Write-only audit service used by all modules; audit viewer	SEC-09
F-19 Shop settings	Business profile, invoice templates, tax rates, master lists, security options, backups and restore	SET-01 to 06, FIN-07
F-16 Branches (core)	Branch create/edit, plan limit check, branch switcher, per-branch access	BR-01, 03, 05
F-15 Staff and payroll	Commission rules engine, shifts, attendance, payroll runs, payslips, salary advances	HR-02 to 05

Screens: Login, Roles and permissions.

Provides to others: Permission service, tenant context, audit service, branch context — used by every other developer.

Developer 2 — Account portal, billing and platform admin

Everything the shop owner does before and around the shop app, the Cellivo super-admin dashboard, and the shared UI component library.

Component	What to build	SRS reference
Shared UI library	Design system components from the SRS/FRS UI section: buttons, cards, stat cards, hero, glass tiles, pills, tables, tabs, inputs, modals, sidebar, layouts, skeletons	UI design section

Component	What to build	SRS reference
F-01 Registration and trial	Sign-up form, email/SMS verification, trial start, setup wizard	ACC-01 to 04
F-02 Subscription and billing	Plan purchase, payment gateway, renewals, retries, upgrade/downgrade, cancel, suspend, invoices	ACC-05 to 10
F-20 Account portal home	Welcome block, hero states, billing setup progress, stat cards, affiliate banner, account menu	ACC-11 to 18
Plan limits service	Feature flags per plan and branch/user limits, checked on the server	2.3, 6
A-01 to A-16 Admin dashboard	Overview KPIs, tenants, tenant actions, support log-in, plans, payments, coupons, SMS credits, leads, affiliates, CMS, tickets, announcements, admin users, system health	ADM-01 to 31
Public website hooks	Pricing page to sign-up hand-off, contact and demo forms into leads	WEB-03 to 06

Screens: Sign-up, Account portal (new and active), Admin overview, tenants, tenant profile, plans.

Provides to others: Shared UI library (needed by all from week 3), plan-limit service, tenant provisioning (with Developer 1).

Developer 3 — POS, sales and money

The counter: selling, taking payment, returns, credit, loyalty redemption, cash drawer and the finance ledger.

Component	What to build	SRS reference
F-09 POS sale	Cart, barcode/IMEI scan, discounts with approval, split payments, hold/void, salesperson, receipts	POS-01 to 06, 11 to 15
Printing	Thermal 58/80 mm and A4/A5 invoice templates, cash drawer kick, reprint	POS-11, INT-04
F-10 Returns, trade-ins, wholesale	Credit notes, exchanges, trade-in intake into stock, wholesale prices	POS-07, 08, 10
F-11 Credit and installments	Credit limits, installment schedules, collections, aging	POS-09, CRM-03 to 05
F-14 Cash drawer and finance	Drawer sessions and Z-report, bank accounts, cheques, expenses, other income, bank reconciliation	FIN-01 to 07
Ledger service	Double-entry postings for sales, purchases, payments, expenses (used by Developers 4 and 5)	FIN-04, 7 data integrity

Component	What to build	SRS reference
Loyalty (earn/redeem)	Points calculation at sale, redemption as payment, reversal on return	CRM-06

Screens: POS billing, Payment, Cash drawer close.

Provides to others: Ledger/payment service (needed by purchasing and repairs invoices), sale events for commissions and reports.

Developer 4 — Inventory, IMEI and purchasing

Everything about stock: products, IMEI life history, purchasing and suppliers, stock transfers, and WooCommerce sync.

Component	What to build	SRS reference
F-06 Products and inventory	Products and variants, CSV import, adjustments, stock count, costing, low-stock alerts	INV-01, 05 to 11
F-07 IMEI tracking	IMEI validation (Luhn), status model, reservation lock, life-history timeline, IMEI search	INV-02 to 04, POS-02
Barcode labels	Barcode generation and label printing	INV-06
F-08 Purchasing and suppliers	Suppliers, purchase orders, GRN with IMEI scan, supplier bills and payments, purchase returns	PUR-01 to 06
F-16 Stock transfers	Request, approve, dispatch, receive; in-transit tracking	BR-02
Stock service	Single place that changes stock; used by POS, repairs parts, returns, transfers	6 design constraints
INT-01 WooCommerce sync	Product, stock and order sync with retry	INT-01

Screens: Inventory, IMEI search and history, GRN.

Provides to others: Stock service and IMEI lookup API — POS (Developer 3) and repairs (Developer 5) depend on it by week 9.

Developer 5 — Repairs, customers, notifications and reports

The repair workflow, customer records, SMS/email, the shop dashboard and all reports.

Component	What to build	SRS reference
F-12 Repairs	Repair intake, job card, technician assignment, stages board, parts used, approval, repair invoice, warranty, overdue	REP-01 to 11

Component	What to build	SRS reference
Public repair status page	Status link / ticket + phone lookup	REP-08
F-13 Customers	Profiles, history, devices owned, duplicates by phone, CSV import, loyalty setup	CRM-01, 02, 06, 07
F-18 Notifications	Notification service (SMS gateway, email, in-app), templates with merge fields, SMS credits deduction, delivery status	INT-02, 03, 06, 07
F-05 Shop dashboard	KPI cards, hero, chart, alerts, role-based dashboards, real-time refresh	DSH-01 to 09
F-17 Reports	All reports, PDF/Excel export, background exports, scheduled emails, Lite plan limits	RPT-01 to 10

Screens: Dashboard, Repair board, New repair, Customer profile, Reports, Mobile views.

Provides to others: Notification service — used by sign-up, POS receipts, repairs and billing emails (needed by week 8).

3. Timeline by developer

Aligned with the 32-week schedule in the SRS (section 8).

Weeks	Developer 1	Developer 2	Developer 3	Developer 4	Developer 5
Wk 1–3	Discovery, data model, API contracts, CI/CD, tenancy skeleton	Shared UI library, design tokens	Data model for sales, payments, ledger	Data model for products, IMEI, stock	Data model for repairs, customers; notification service design
Wk 4–8	Login, 2FA, roles and permissions, audit log	Sign-up, trial, subscriptions, payment gateway, portal home	POS cart and payments (using stub stock API), printing	Products, IMEI, stock service, CSV import, labels	Notification service, customers, repair intake
Wk 9–15	Shop settings, branches core, backups	Admin dashboard: overview, tenants, plans, payments, coupons	Returns, trade-ins, credit, installments, loyalty	Purchasing, GRN, suppliers, stock adjustments and count	Repair workflow, parts, invoices, status page
Wk 16–21	Staff, commissions, attendance	Admin: leads, affiliates, CMS, tickets, announcements	Cash drawer, banks, cheques, expenses, ledger	Stock transfers, multi-branch stock	Dashboard, reports part 1
Wk 22–26	Payroll, payslips; security hardening	Admin: system health, feature flags, SMS credits	Bank reconciliation, finance reports data	WooCommerce sync	Reports part 2, exports, scheduled reports
Wk 27–32	Performance, load tests, go-live, data migration	Bug fixing, UAT support	Bug fixing, UAT support	Bug fixing, UAT support	Bug fixing, UAT support

4. Dependencies between developers

These shared services must be ready on time because other developers build on them. Until then, the owner provides a stub (fake) version with the agreed API so nobody is blocked.

Shared service	Built by	Used by	Needed by
Permission and tenant context	Developer 1	All	Week 4
Shared UI component library	Developer 2	All	Week 3 (first set), full by week 6
Stock service and IMEI lookup API	Developer 4	Developers 3 and 5	Week 9 (stub API by week 4)
Ledger / payment posting service	Developer 3	Developers 4 and 5	Week 10
Notification service (SMS, email, in-app)	Developer 5	Developers 2, 3, 4	Week 8
Plan-limit and feature-flag service	Developer 2	All	Week 6
Commission hooks (sale and repair events)	Developers 3 and 5	Developer 1	Week 16
Sales, stock and repair data for reports	Developers 3, 4, 5	Developer 5	Week 18

5. Code review pairs

Every pull request needs one approval. The second reviewer covers when the first is away and is the backup owner of that area, so knowledge is never held by one person only.

Author	First reviewer	Backup reviewer / backup owner
Developer 1	Developer 2	Developer 4
Developer 2	Developer 1	Developer 3
Developer 3	Developer 4	Developer 5
Developer 4	Developer 3	Developer 1
Developer 5	Developer 3	Developer 2

6. Team rules

Working together

- Two-week sprints; planning on day 1, demo and retrospective on the last day.
- 15-minute daily stand-up: done yesterday, doing today, blocked by.
- Agree API contracts (request/response format) in writing before building a shared service.
- One task board; every ticket carries its FRS function ID and SRS requirement IDs.
- Developer 1 approves database schema changes and anything touching tenant isolation or security.

Code

- Branch per ticket (e.g. feature/F-09-split-payments); merge to main only through reviewed pull requests.
- Only the stock service changes stock and only the ledger service posts money; no module writes those tables directly.
- Every query goes through the tenant context; automated tests check one tenant cannot see another's data.
- Use only components from the shared UI library; new components are added to the library, not copied.
- Money stored as fixed-point decimals; all dates stored in UTC.

Definition of done

- Meets the FRS acceptance criteria for that function.
- Unit tests for business logic; at least 70% coverage in the module.
- Permissions and plan limits checked on the server.
- Actions written to the audit log where required.
- Works on desktop, tablet and phone widths and matches the UI design section.
- Reviewed, merged and deployed to staging; QA has tested it.

7. Risks and how to handle them

Risk	Mitigation
POS (Developer 3) blocked waiting for stock service	Developer 4 delivers a stub stock API in week 4 and the real one by week 9.
Developer 5 has the most screens (dashboard and reports come last)	Reports start after data exists (week 16); Developers 2 and 4 help with reports in weeks 22–26 if behind.
Developer 1 overloaded as tech lead	Payroll can move to Developer 2 after week 20 if needed.
Payment and SMS provider not chosen	Client to confirm providers by week 3 (see SRS open items).
One developer leaves or is sick	Backup owner from section 5 takes over; keep README and API notes per module.

Cellivo development work allocation v1.0. Update this document when ownership or dates change.